

Trabalho 4: Aprendizado Associativo

1st Nycksandro Lima dos Santos
Instituto de Computação
Universidade Federal do Amazonas
Manaus, Brasil
nycksandro.santos@icomp.ufam.edu.br

Abstract—Este trabalho apresenta da resolução do exercício 15.9 do livro *Neural Network Design 2nd Edition*

I. PROBLEMA

A rede mostrada na Figura 1 está instalada em um elevador usado por três executivos seniores em um luxuoso prédio corporativo de alta segurança. Ela possui botões marcados de ‘1’ a ‘4’ para quatro andares acima do térreo. Quando um executivo entra no elevador no térreo, o sistema determina qual pessoa é através de uma leitura da retina e, em seguida, usa a rede para selecionar o andar para onde essa pessoa provavelmente irá. Se a previsão estiver incorreta, a pessoa pode pressionar um botão diferente a qualquer momento, mas se a rede estiver correta, isso evitará que um executivo importante precise pressionar um botão [1].

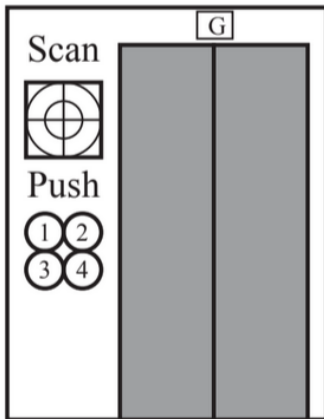


Fig. 1. Ilustração do elevador descrito

A. Simule a rede para a seguinte sequência de eventos:

- 1) Presidente pressiona ‘4’
- 2) Vice-Presidente pressiona ‘3’
- 3) Chairman pressiona ‘1’
- 4) Vice-Presidente pressiona ‘3’
- 5) Chairman pressiona ‘2’
- 6) Presidente pressiona ‘4’

A simulação foi realizada utilizando o seguinte código (versão simplificada) em Python. Os detalhes da implementação estão disponíveis em: Nycksandro_Lima_Trabalho04.

Implementação da função hardlims

```
def hardlims(x):  
    return np.where(x >= 0, 1, -1)  
  
# Implementação da função Outstar  
  
def outstar(a, entrada, w_antigo, taxa_aprendizado):  
    erro = a - (w_antigo @ entrada)  
    w_novo = w_antigo + taxa_aprendizado * (erro @ entrada.T)  
    return w_novo  
  
# Definição da função de treinamento  
  
def treinamento(peso_inicial, ordem_botoes, taxa_aprendizado, vies):  
    peso_atual = peso_inicial  
  
    for i, botao in enumerate(ordem_botoes, start=1):  
        retina, andar = botao  
  
        if (andar is not None):  
            # Verificando se foi apertado nenhum botão  
            n = (peso_atual @ retina) + (peso_atual @ retina) + vies  
            a = hardlims(n) # Calculando a saída a  
  
            # Atualizando o peso usando outstar  
            peso_atual = outstar(a, retina, peso_atual, taxa_aprendizado)  
  
        else:  
            # Caso não for, aqui o código do elevador será nulo (p0 = 0)  
            n = (peso_atual @ retina) + vies  
            a = hardlims(n)  
  
    return peso_atual
```

O retorno da simulação teve a seguinte saída:

• Passo 1

– Saída calculada (a): $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$

– Matriz de Pesos Atualizada (W): $\begin{bmatrix} 0.6 & 0 & 0 \\ 0.6 & 0 & 0 \end{bmatrix}$

• Passo 2

– Saída calculada (a): $\begin{bmatrix} -1 \\ 1 \end{bmatrix}$

– Matriz de Pesos Atualizada (W): $\begin{bmatrix} 0.6 & -0.6 & 0 \\ 0.6 & 0.6 & 0 \end{bmatrix}$

• Passo 3

– Saída calculada (a): $\begin{bmatrix} -1 \\ -1 \end{bmatrix}$

– Matriz de Pesos Atualizada (W): $\begin{bmatrix} 0.6 & -0.6 & -0.6 \\ 0.6 & 0.6 & -0.6 \end{bmatrix}$

• Passo 4

– Saída calculada (a): $\begin{bmatrix} -1 \\ 1 \end{bmatrix}$

– Matriz de Pesos Atualizada (W): $\begin{bmatrix} 0.6 & -0.84 & -0.6 \\ 0.6 & 0.84 & -0.6 \end{bmatrix}$

• Passo 5

– Saída calculada (a): $\begin{bmatrix} -1 \\ -1 \end{bmatrix}$

– Matriz de Pesos Atualizada (W): $\begin{bmatrix} 0.6 & -0.84 & -0.84 \\ 0.6 & 0.84 & -0.84 \end{bmatrix}$

• Passo 6

- Saída calculada (a): $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$
- Matriz de Pesos Atualizada (W): $\begin{bmatrix} 0.84 & -0.84 & -0.84 \\ 0.84 & 0.84 & -0.84 \end{bmatrix}$

B. Quais são os pesos finais?

Ao fim do passo 6, a matriz de pesos se encontra da seguinte maneira:

$$\begin{bmatrix} 0.84 & -0.84 & -0.84 \\ 0.84 & 0.84 & -0.84 \end{bmatrix}$$

C. Agora, continue simulando a rede nesses eventos:

- 1) Presidente não aperta um botão
- 2) Vice-Presidente não aperta um botão
- 3) Chairman não aperta um botão.

A saída da simulação para esses eventos é:

• Passo 7

- Saída calculada (a): $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$
- Matriz de Pesos Atualizada (W): $\begin{bmatrix} 0.84 & -0.84 & -0.84 \\ 0.84 & 0.84 & -0.84 \end{bmatrix}$

• Passo 8

- Saída calculada (a): $\begin{bmatrix} -1 \\ 1 \end{bmatrix}$
- Matriz de Pesos Atualizada (W): $\begin{bmatrix} 0.84 & -0.84 & -0.84 \\ 0.84 & 0.84 & -0.84 \end{bmatrix}$

• Passo 9

- Saída calculada (a): $\begin{bmatrix} -1 \\ -1 \end{bmatrix}$
- Matriz de Pesos Atualizada (W): $\begin{bmatrix} 0.84 & -0.84 & -0.84 \\ 0.84 & 0.84 & -0.84 \end{bmatrix}$

D. A que andares a rede levou cada executivo?

TABLE I
ESCOLHA DOS EXECUTIVOS VS PREVISÃO DA RETINA

Iteração	Executivo	Botão Pressionado	Destino Final (Elevador)
1	Presidente	4	4
2	Vice-Presidente	3	3
3	Chairman	1	1
4	Vice-Presidente	3	3
5	Chairman	2	2
6	Presidente	4	4
7	Presidente	Nenhum	4
8	Vice-Presidente	Nenhum	3
9	Chairman	Nenhum	1

E. Se os executivos pressionassem os seguintes botões várias vezes, como você esperaria que fosse a matriz de pesos resultante?

- 1) Presidente pressiona '3'
- 2) Vice-presidente pressiona '2'
- 3) Chairman pressiona '4'

Ao apertar diversas vezes os mesmos andares, a matriz de pesos irá convergir para o valor exato do andar desejado por cada executivo.

A matriz de pesos aos poucos irá se alterar até ficar assim:

Matriz de pesos:

$$\begin{bmatrix} -1 & 1 & 1 \\ 1 & -1 & 1 \end{bmatrix}$$

O Notebook do Colab deste trabalho está disponível na URL a seguir: Nycksandro_Lima_Trabalho04

REFERENCES

- [1] M. T. Hagan, H. B. s. Demuth, M. H. Beale, and O. De Jesús, *Neural Network Design*, 2nd ed. Martin Hagan, 2014.